

CampusClaw: Human-in-the-Loop Agentic Actions for Explainable Campus Matching

Zhicheng Xu, Zhen Yang, Kaixuan Wen, Zeshuai Zhu, Houxin Chang

Nankai University, Tianjin, China

{xzc, yz, wx, zzs, chx}@mail.nankai.edu.cn

Abstract

Campus communities depend on fragmented channels such as group chats, bulletin boards, informal marketplaces, and social feeds to support team formation, peer-to-peer trading, dating, and information exchange. Large language models (LLMs) can reduce the friction of form filling, search, and first-contact communication, but directly connecting an LLM to application APIs may introduce unsafe execution, hallucinated recommendations, and unclear user responsibility. This paper presents CampusClaw, a working campus application that explores a human-in-the-loop agentic interaction pattern for localized matching. CampusClaw converts natural-language requests into schema-grounded action proposals, validates them through a backend skill registry, and lets users choose whether to execute, only fill a draft, or only keep the generated suggestion. Its recommendation module returns only real database objects and attaches scores and explanations for team recruitment, dating, and second-hand trading. We describe the design process, system architecture, action proposal pipeline, explainable matching logic, and preliminary user feedback. CampusClaw demonstrates how LLM-based assistance can be embedded into campus applications without giving the model unconstrained authority over user data or application state.

1 Introduction

University students frequently need to find teammates, trade second-hand items, ask for campus information, or initiate social connections. In practice, these needs are scattered across WeChat groups, forums, campus marketplaces, and informal social channels. The result is not simply an absence of information, but a mismatch between user intent and the fragmented places where useful opportunities appear. Students must repeatedly search, filter, compare, and decide how to start a conversation.

LLMs provide a promising interface for such scenarios. A student can express a goal in natural language, e.g., “recommend several frontend teammates for an AI competition” or “help me write a polite first message to contact the seller.” However, deploying LLMs inside interactive social applications raises three risks. First, an LLM should not be allowed to freely call arbitrary backend APIs. Second, a generative model may fabricate users, posts, or products if recommendations are produced as free text. Third, campus matching is socially sensitive: users often want assistance, but they still want final control over what is published, sent, or saved.

We address these issues through **CampusClaw**, a campus-oriented social and collaboration platform. The implemented application contains forum posting, team recruitment, dating matching, trade matching, messaging, profile management, reporting, blocking, and an AI assistant. The key design idea is to treat the LLM as a *proposal generator* rather than an autonomous operator. The model helps interpret user

intent and draft structured payloads, while the backend validates each action against a whitelist of skills and the user decides whether to execute the action.

This paper contributes:

- A schema-grounded action proposal pipeline that turns natural language into validated, user-confirmed application actions.
- An explainable recommendation mechanism that only recommends real database objects and presents scores and reasons.
- A human-in-the-loop interaction design with three choices: fill and publish, fill only, or generate only.
- A working campus application prototype that integrates LLM assistance into multiple concrete scenarios rather than a standalone chatbot.

2 Design Motivation and Requirements

The design of CampusClaw follows a human-centered process rather than starting from model capability alone. In the project presentation and prototype study, we focused on four recurring campus connection scenarios: finding team members, finding dating or interest-based matches, trading second-hand goods, and browsing campus forum information.

2.1 User Research

We conducted a small competition-oriented user research process with students aged 18–25 who had experience with project teams, campus activities, second-hand trading, or campus forums. The study combined survey questions, informal interviews, and task-based prototype use. The product presentation reports four pain points: fragmented information channels, time-consuming filtering, discomfort when initiating contact, and missed time-sensitive opportunities. In the preliminary survey summary, 82% of respondents reported information fragmentation,

76% reported filtering fatigue, 68% reported embarrassment or pressure when contacting strangers, and 71% reported missing opportunities because information arrived too late.

These values are not intended as a large-scale statistical claim. Instead, they motivated the product direction: a system should not only aggregate campus content, but also explain why a connection is relevant and reduce the pressure of first contact.

2.2 Requirements

The observations led to four requirements.

R1: Unified opportunity discovery. The application should aggregate forum posts, team recruitment, dating candidates, and trade posts into a coherent product flow.

R2: Explainable matching. A recommendation should not only output a target, but also explain why it is relevant, e.g., shared skills, matched interests, or keyword overlap.

R3: Human-controlled automation. AI should reduce writing and search effort, but final publication or message sending should remain under user control.

R4: Safe backend execution. The LLM should not directly construct URLs or freely invoke backend operations. All executable actions should be routed through a backend registry with validation.

These requirements are relevant for semantic web and knowledge-enabled systems: the core challenge is not merely generating fluent text, but grounding agentic behavior in structured entities, typed actions, and explainable relationships.

3 System Overview

CampusClaw is implemented as a uni-app/Vue3 client and a Django REST backend. The client can be packaged as a WeChat mini-program or an Android APK. The backend provides modules for accounts, profiles, forum posts, team recruitment, dating profiles and preferences, trade posts, chat messages, moderation, and assistant actions.

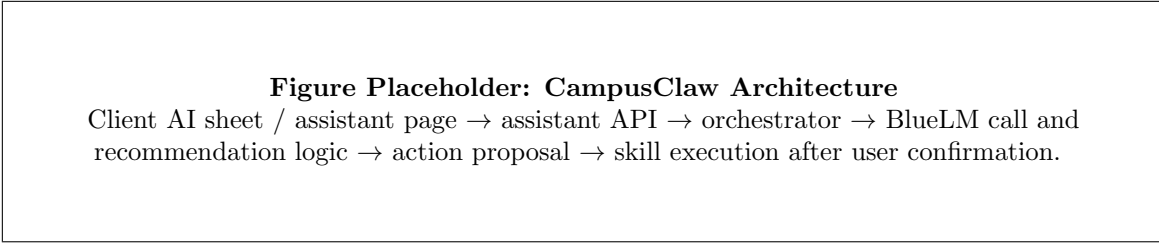


Figure 1: CampusClaw uses the LLM as a bounded reasoning component. The backend orchestrator creates proposals, and the skill registry executes only validated actions after explicit user confirmation.

The assistant subsystem is organized around four backend components: `assistant/agent.py` for BlueLM-compatible chat completion calls, `assistant/orchestrator.py` for turn planning and prompt construction, `assistant/recommendations.py` for database-grounded recommendation, and `assistant/skills.py` for executable action registration. On the frontend, `services/assistant.ts`, `AssistantSheet.vue`, and `AssistantActionCard.vue` handle assistant requests, half-screen interaction, and proposal-card rendering. This structure separates model invocation, orchestration, business execution, and client rendering.

4 Schema-Grounded Action Proposals

The core interaction primitive is an **AssistantActionProposal**. Instead of returning only a natural-language answer, the system may return a structured action card with a type, title, target page, preview, fill payload, status, and expiration time.

4.1 Pipeline

The proposal pipeline consists of five steps.

Step 1: Intent interpretation. When a user sends a message, the backend routes it to `plan_assistant_turn()`. The orchestrator first checks whether the prompt is a recommendation or

icebreaker request. Otherwise, it asks the LLM to infer the user’s intent and missing fields.

Step 2: Payload generation. The orchestrator constructs prompts that include the current page type, context path, recent history, allowed intent names, and allowed fields. The model is asked to return structured JSON rather than an unconstrained command.

Step 3: Backend validation. The returned fields are sanitized and normalized. Only fields allowed for the selected intent are kept. For example, a team recruitment action may include title, summary, details, target size, tags, and required skills; a trade action may include post type, title, description, price, condition, tags, and negotiability.

Step 4: User-facing proposal. The frontend renders an action card. The user can choose “fill and publish”, “fill only”, or “generate only”. This preserves user control and supports low-risk previewing.

Step 5: Skill execution. If the user confirms execution, the backend checks ownership, status, expiration, and skill registration before creating a post, submitting an application, favoriting a trade post, or sending a chat message.

4.2 Safety Boundary

The system explicitly rejects a direct tool-use pattern in which the model freely selects arbitrary APIs. The LLM proposes; the backend validates; the user confirms; the registry executes. High-risk actions such as deleting content, blocking users, hiding conversations, or bypassing moderation are not exposed as assistant skills. This does not make the system im-

Table 1: Examples of schema-grounded AI actions in CampusClaw.

Action Kind	Structured Payload	User-Controlled Outcome
Forum post creation	title, body, category, tags	Fill draft or publish after confirmation
Forum interaction	post/comment identifier, body or like signal	Comment, reply, or like after confirmation
Team recruitment	title, summary, details, target size, skills	Create an open team post or fill an application
Trade post creation	type, title, description, price, condition, tags	Create a trade listing or fill a draft
Dating signal	target user, interested/skip signal	Express interest or skip after confirmation
Context chat message	target user, source type, message body	Send a confirmed private message

immune to every prompt-injection risk, but it narrows the executable surface to typed, auditable, and user-confirmed actions.

5 Explainable Matching and Recommendation

CampusClaw includes three recommendation scenarios: team recruitment, dating matching, and trade matching. The system deliberately separates retrieval from generation. Recommended objects must already exist in the database; the LLM is not allowed to invent a user, team, or product.

Team recommendation. The function `build_team_recommendations()` uses the current user prompt as the main signal. It matches prompt terms against team post titles, summaries, details, tags, and required skills. It filters out the user’s own posts, closed posts, and posts whose current size has reached the target size. The result includes a recommendation score and a reason such as keyword or skill overlap.

Dating recommendation. The function `build_dating_recommendations()` combines saved dating preferences with the current prompt. Hard preference filters, such as age, height, weight, and gender ranges, are applied unless the user explicitly relaxes them. The current prompt is used as a temporary ranking signal, for example preferring candidates

Table 2: Recommendation grounding policies.

Scenario	Grounding and Explanation
Team	Real open posts; explanation based on skills, tags, and text overlap
Dating	Visible profiles; explanation based on saved preferences and current prompt
Trade	Open trade posts; explanation based on item keywords and trade metadata

who mention photography or similar interests.

Trade recommendation. The function `build_trade_recommendations()` focuses on item and transaction keywords in the user’s prompt. It matches title, description, condition, tags, and trade type, filters out the user’s own posts, and ranks open trade posts. The output supports viewing details, favoriting, or contacting the seller.

The recommendation card includes a title, subtitle, score, score label, reason, and next-step buttons such as contact, fill application, favorite, interested, or skip. This makes the system closer to explainable semantic matching than to a black-box chatbot response. It also creates a bridge to future semantic web extensions: users, posts, skills, interests, trade items, and actions can be modeled as typed entities and relations rather than unstructured strings.

Figure Placeholder: AI Action Card
Buttons: Fill and publish / Fill only /
Generate only

Figure 2: The action card exposes automation levels explicitly. Users may execute the backend action, fill a draft for later editing, or keep the generated text only.

6 Human-in-the-Loop Interaction

CampusClaw’s assistant is accessible through a half-screen sheet and an independent assistant page. The half-screen design allows the user to ask for help while staying on the current page. This is important for contextual tasks: a user viewing a team post may ask for an application message, while a user viewing a trade post may ask for an icebreaker message.

The three user choices represent different levels of automation:

- **Fill and publish:** the user confirms the preview and the backend executes the corresponding skill.
- **Fill only:** the generated payload is stored as a pending draft and loaded into the target form, allowing further editing.
- **Generate only:** the assistant reply remains as text, with no change to application state.

This design is especially useful for socially sensitive interactions. Users may accept an AI-generated icebreaker only after seeing the full text. The model can reduce anxiety, but the user still decides whether to send. In the dating scenario, low-pressure actions such as “interested” and “skip” are separated from direct chat initiation. In team and trade scenarios, contact suggestions and draft messages are shown before execution.

7 Preliminary Evaluation

CampusClaw is currently positioned as a working competition prototype. We therefore report preliminary, small-scale feedback rather than a formal deployment study.

The prototype evaluation involved task-based use across three representative flows: (1) ask AI to recommend team posts and submit a team application, (2) ask AI to recommend a trade item and contact the seller, and (3) use AI to generate a first-contact message. We observed whether users could complete the task, whether they understood why an item was recommended, and whether they preferred direct execution or draft filling.

The product presentation reports a small usability test with 28 participants. In that internal test, AI-generated icebreaker messages were associated with a higher reply rate in prototype scenarios, and AI-filled application drafts were associated with higher application completion. Because the test was small and competition-oriented, we treat these numbers as early evidence for design usefulness, not as a generalizable causal result.

Qualitative feedback was consistent with the design requirements. Users reported that recommendation reasons made matches feel less random, team application cards were clearer than searching through chat groups, and AI-generated opening messages reduced the difficulty of starting a conversation. During iteration, we fixed concrete interaction issues including input layout, search result display, contact buttons, image rendering on real devices, and clearer Chinese wording.

8 Discussion

CampusClaw illustrates a middle path between traditional GUI applications and fully autonomous agents. It does not replace structured application logic with free-form model behavior. Instead, it places LLM output inside a typed, inspectable, and revocable proposal workflow.

For semantic web and knowledge-enabled systems, this architecture suggests two directions. First, cam-

pus entities such as users, skills, posts, interests, trade items, and actions can be represented as a lightweight domain schema. Second, future versions may replace simple keyword and attribute overlap with ontology-aware or knowledge-graph reasoning, e.g., inferring that “data visualization” relates to D3.js, dashboard design, and frontend development.

There are limitations. The current prototype uses relational database rows and hand-written matching functions rather than a formal ontology. The evaluation is small-scale and competition-oriented. Production deployment would require stronger security policies, privacy review, HTTPS deployment, domain registration, and larger-scale field testing. The current system should therefore be read as a design and engineering probe into safe agentic campus applications, not as a finished campus-wide infrastructure.

9 Conclusion

We presented CampusClaw, a campus matching and collaboration application that integrates LLM assistance through schema-grounded, human-in-the-loop action proposals. The system uses BlueLM to understand intent, generate drafts, recommend real database objects, and produce contact suggestions, while backend validation and user confirmation prevent unconstrained execution. The prototype demonstrates how agentic interaction can be embedded into a practical campus application without turning the LLM into an unsafe autonomous operator. Future work will focus on stronger semantic modeling, knowledge-graph-based matching, and larger-scale user evaluation.

References

- [1] Brown, T.: Design thinking. *Harvard Business Review* 86(6), 84–92 (2008).
- [2] Berners-Lee, T., Hendler, J., Lassila, O.: The Semantic Web. *Scientific American* 284(5), 34–43 (2001).
- [3] Hogan, A., Blomqvist, E., Cochez, M., d’Amato, C., de Melo, G., Gutierrez, C., Kirrane, S., Gayo, J.E.L., Navigli, R., Neumaier, S., et al.: Knowledge Graphs. *ACM Computing Surveys* 54(4), 1–37 (2021).
- [4] Floridi, L., Cows, J., Beltrametti, M., Chatila, R., Chazerand, P., Dignum, V., Luetge, C., Madelin, R., Pagallo, U., Rossi, F., et al.: AI4People—An ethical framework for a good AI society. *Minds and Machines* 28(4), 689–707 (2018).
- [5] Xi, Z., Chen, W., Guo, X., He, W., Ding, Y., Hong, B., Zhang, M., Wang, J., Jin, S., Zhou, E., et al.: The rise and potential of large language model based agents: A survey. *arXiv preprint arXiv:2309.07864* (2023).
- [6] Qin, Y., Liang, S., Ye, Y., Zhu, K., Lu, C., Lin, Y., Jindal, P., Wang, X., Yuan, Z., Sanwal, G., et al.: Tool learning with foundation models. *arXiv preprint arXiv:2304.08354* (2023).
- [7] Zanzotto, F.M.: Human-in-the-loop artificial intelligence. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery* 10(6), e1382 (2020).